Implementation Plan

RMC-Spark: Resilient Multi-Cluster Spark Analytics

for the paper "Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments"

Team ID	26FSPDSCPMID4098
Members	26MML0054 · 26MML0006 · 26MML0047 · 26MML0015 (PoC)
Course	Programming for Data Science (MACSE502), SCOPE, VIT
Repository	rmc-spark/ (reference implementation already written; 56 automated tests passing)
Duration	8 weeks (a 3-week compressed schedule is given in Section 8)
Version	October 2026

How to use this plan. The system is built in eight phases, P0 to P7. Each phase lists its goal, owner, tasks, the files that implement it, the commands to run and a 'done when' checklist. A reference implementation of every file already exists in `rmc-spark/`, so for each phase you can either study and run the existing module or rewrite it yourself following the specification in Section 5, then prove it with the same tests and checks. Do not move to the next phase until the 'done when' list of the current one is complete.

1. What We Are Building 3
2. Technology Stack 4
   Hardware budget (16 GB laptop) 4
3. Environment Setup (Phase P0 details) 5
   Repository layout and owners 5
4. Build Phases 6
   P0 — Environment, repository and team setup 6
   P1 — Testbed: two Spark clusters, storage and monitoring 6
   P2 — Data and workloads 7
   P3 — rds library: resilient, resumable stages 8
   P4 — Resilience controller (C1–C5) 8
   P5 — Fault injection (chaos harness) 9
   P6 — Experiments 9
   P7 — Analysis, figures, paper integration and deliverables 10
5. Module Specifications 11
   Key configuration (configs/clusters.yaml) 11
6. Testing Strategy 13
7. Experiment Protocol 14
   Expected results to check (hypotheses) 14
8. Timeline and Responsibilities (8 weeks) 15

Compressed 3-week schedule (if the deadline is near) 15
9. Risks and Troubleshooting 16
10. Definition of Done and Deliverables 17
Command cheat sheet 17

1. What We Are Building

RMC-Spark is a resilience layer above two (or more) unmodified Apache Spark clusters. A controller watches every cluster, scores its health, and when a cluster fails it moves stalled jobs to a healthy cluster. A small PySpark library, rds, makes every pipeline stage idempotent and commits it to shared object storage, so the moved job restores finished stages instead of recomputing them. Every decision is logged with its cause.

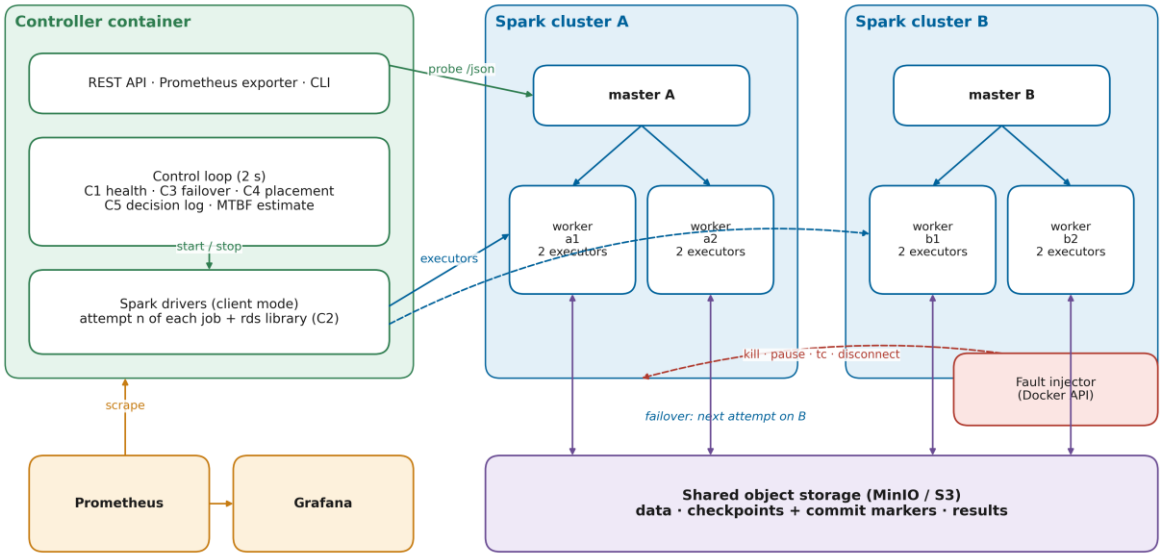


Figure 1. Target architecture (Fig. 6 of the paper).

Component	Purpose	Implemented in
Testbed	2 Spark clusters (1 master + 2 workers each), MinIO, Prometheus, Grafana, controller	docker-compose.yml, docker/
rds library (C2)	Resumable stages, commit markers, adaptive (Young/Daly) checkpointing, event log	rds/
Controller C1	Probes, EWMA anomaly detection, health score, HEALTHY/SUSPECT/FAILED state machine	controller/monitor.py, anomaly.py, health.py, statemachine.py
Controller C3	spark-submit drivers in client mode; retry, migrate or keep decisions	controller/jobs.py, core.py
Controller C4	Placement cost $J = \alpha L + \beta(1-H) + \gamma T$	controller/orchestrator.py
Controller C5	Decision log, reliability score R, REST API, Prometheus metrics	controller/decisions.py, reliability.py, api.py, cli.py
Workloads	ETL, ML (GBT), iterative PageRank, streaming, 2 case studies	jobs/
Fault injection	9 faults via Docker API (kill, partition, tc delay, CPU throttle, disk fill, driver crash)	chaos/inject.py
Experiments & analysis	Matrix runner, metrics, statistics, 600-DPI figures and tables	experiments/, analysis/

2. Technology Stack

Layer	Technology (version)	Why
Containers	Docker Desktop 28 + Compose v2 (WSL2, 12 GB)	Several real clusters on one laptop; faults are real container operations
Compute	Apache Spark 3.5.3 standalone, official `apache/spark` image, Java 17	Stable; PySpark included; Prometheus metrics servlet
Language	Python 3.10 in containers (3.10+ on host for tests)	Course focus: data science programming in PySpark
Storage	MinIO (S3-compatible) via Hadoop S3A 3.3.4 + AWS SDK 1.12.262	Shared checkpoints readable by every cluster
Monitoring	Prometheus + Grafana (provisioned dashboard)	Operator view and screenshots/videos
Controller	FastAPI, uvicorn, requests, prometheus-client, PyYAML, docker SDK	API, probes, metrics, fault injection
Analysis	pandas, numpy, scipy (Mann-Whitney U), matplotlib (600 DPI)	Tables 11–15, Figures 21–24
Testing	pytest (56 tests: unit + controller simulation)	Logic verified without Docker
Paper tooling	Anaconda Python (lxml, matplotlib), Node `docx`	Figure generation and manuscript build

Hardware budget (16 GB laptop)

Container	Memory limit	Count	Total
Spark master	768 MB	2	1.5 GB
Spark worker (2 cores, 1280 MB to executors)	1.75 GB	4	7.0 GB
Controller (hosts the drivers)	2.5 GB	1	2.5 GB
MinIO / Prometheus / Grafana	512 / 384 / 256 MB	3	1.15 GB
Total			≈ 12.2 GB → WSL2 memory = 12 GB

3. Environment Setup (Phase P0 details)

1. Install **Docker Desktop**, **Git**, **VS Code** (Python + Docker extensions), **Anaconda** (or Python 3.10+).
2. Create `C:\Users\<you>\.wslconfig`:

```
[wsl2]
memory=12GB
processors=10
```

1. Run `wsl --shutdown`, restart Docker Desktop, check `docker info` (Total Memory ≈ 12 GiB).
2. Clone the repository and run the unit tests (no Docker needed): `python -m pytest` → `56 passed`.
3. First build downloads ≈ 1.5 GB (base image, Maven jars, Python packages); do it on a good connection.
4. Open http://localhost:8080, :8090, :8000/docs, :9001 (MinIO, rmcadmin/rmcadmin123), :9090, :3000 after `docker compose up -d`.

Repository layout and owners

Path	Content	Owner
docker-compose.yml, docker/	Testbed definition, Spark config, Prometheus, Grafana	26MML0006
configs/clusters.yaml	Cluster registry + every controller threshold	26MML0015
rds/	Resilient stage library	26MML0006
controller/monitor, anomaly, health, statemachine, reliability, decisions	C1, C5	26MML0054
controller/jobs, core, orchestrator, api, cli, workloads	C3, C4, integration	26MML0015
jobs/, scripts/, datasets/	Workloads, data generation, case studies	26MML0006 / 26MML0047
chaos/	Fault injection	26MML0006
experiments/, analysis/	Matrix runner, metrics, statistics, figures	26MML0006 / 26MML0054
tests/	Unit and simulation tests	module owner
paper/	Manuscript sources, figures, build scripts, guides	26MML0047 + all

4. Build Phases

Phases are sequential; within a phase, tasks of different owners can run in parallel. Each phase ends with a short demo to the team and a merged pull request.

Phase	Title	When	Owner
P0	Environment, repository and team setup	Week 1 (days 1–2)	26MML0015 (PoC) + all
P1	Testbed: two Spark clusters, storage and monitoring	Week 1 (days 3–5)	26MML0006
P2	Data and workloads	Week 2	26MML0006 (W1, W3, W4), 26MML0047 (case studies)
P3	rds library: resilient, resumable stages	Week 2–3	26MML0006
P4	Resilience controller (C1–C5)	Week 3–4	26MML0054 (C1, C5), 26MML0015 (C3, C4, core)
P5	Fault injection (chaos harness)	Week 4	26MML0006
P6	Experiments	Week 5–6	26MML0006 (run), 26MML0054 (monitor data quality)
P7	Analysis, figures, paper integration and deliverables	Week 7–8	26MML0054 (analysis), 26MML0047 (paper), 26MML0015 (integration, videos)

P0 — Environment, repository and team setup

When	Week 1 (days 1–2)
Owner	26MML0015 (PoC) + all
Goal	Every member can run the tests and has the repository; GitHub is the single source of truth.
Files	README.md, pytest.ini, .gitignore, tests/

Tasks

1. Install Docker Desktop (WSL2 back-end), Git, VS Code, Python 3.10+ (Anaconda is fine).
2. Create `C:\Users\<you>\.wslconfig` with `memory=12GB` and `processors=10`, then `wsl --shutdown` and restart Docker Desktop.
3. PoC creates a private GitHub repository `rmc-spark`, pushes the `rmc-spark/` folder, adds the three members as collaborators.
4. Everyone clones, creates a branch per task (`feat/<area>-<short-name>`), and runs `python -m pytest` (56 tests must pass).
5. Agree on conventions: Black-compatible formatting, one PR per task, PoC reviews and merges, no direct pushes to `main`.

Commands

```
git clone <repo-url>
cd rmc-spark
python -m pip install pytest requests
python -m pytest
```

Done when

- ☐ All four members see `56 passed`.
- ☐ `docker info` shows ≥ 12 GB memory.
- ☐ Branch protection on `main` enabled.

P1 — Testbed: two Spark clusters, storage and monitoring

When	Week 1 (days 3–5)
Owner	26MML0006
Goal	Two independent Spark 3.5.3 standalone clusters (A, B), MinIO, Prometheus, Grafana and the controller container start with one command.

Files	docker/Dockerfile, docker/spark-defaults.conf, docker/metrics.properties, docker/prometheus.yml, docker/grafana/*, docker-compose.yml

Tasks

1. Build the image from `docker/Dockerfile` (Spark 3.5.3 + hadoop-aws 3.3.4 + AWS SDK 1.12.262 + iproute2/procps + Python libs).
2. Start the stack with `docker-compose.yml`; check that `rmc-minio-init` creates the buckets `data`, `checkpoints`, `results` and exits 0.
3. Verify both Spark UIs show 2 ALIVE workers / 4 cores; verify Prometheus targets are UP and the Grafana dashboard loads.
4. Run a smoke Spark job from the controller against each master (SparkPi or `jobs/datagen.py` with small sizes) to prove driver↔executor connectivity in client mode.
5. Record the resource usage (`docker stats`) and adjust `mem_limit` / worker `--memory` if containers are OOM-killed (exit 137).

Commands

```
docker compose build
docker compose up -d
docker compose ps
docker compose exec controller python -m controller.cli status
```

Done when

- ☐ http://localhost:8080 and :8090 show 2 workers each.
- ☐ http://localhost:8000/metrics shows `rmc_cluster_health{cluster="A"} 1.0`.
- ☐ A Spark job submitted from the controller to master A and to master B finishes.
- ☐ Screenshots saved for Section XIII (Fig. 19).

P2 — Data and workloads

When	Week 2
Owner	26MML0006 (W1, W3, W4), 26MML0047 (case studies)
Goal	All workloads run cleanly on one cluster and their clean runtimes are known.
Files	jobs/datagen.py, jobs/common.py, jobs/etl_taxi.py, jobs/ml_pipeline.py, jobs/iterative_rank.py, jobs/streaming_rate.py, jobs/ingest_local.py, scripts/download_data.py, controller/workloads.py

Tasks

1. Generate the synthetic data with `jobs/datagen.py` (4 M taxi trips, 2 M-edge skewed graph) into `s3a://data/`.
2. Run each workload once without faults: `etl_taxi`, `ml_taxi`, `iterative_rank`, `streaming` (submit through the controller CLI with `--mode none --policy never`).
3. Measure clean runtimes (3 runs each); if a workload takes < 60 s or > 6 min, adjust its size (`--taxi-rows`, `--iterations`, `--sample`, `--target-batches`).
4. Case studies: `python scripts/download_data.py --case-studies`, ingest with `jobs/ingest_local.py`, run `case_fraud` and `case_maintenance`, record AUC.
5. Optional: download one NYC TLC month and run `etl_taxi --source tlc` to show the pipeline works on real data.

Commands

```
docker compose exec controller spark-submit --master spark://spark-master-a:7077 --conf
spark.driver.host=controller --total-executor-cores 4 --executor-memory 576m jobs/datagen.py
docker compose exec controller python -m controller.cli submit etl_taxi --mode none --policy never
```

Done when

- ☐ Each workload SUCCEEDED 3× with stable runtimes (spread < 15 %).
- ☐ A table of clean runtimes exists (needed for `fault_at_s` in P6).

- ☐ Case-study AUC values recorded.

P3 — rds library: resilient, resumable stages

When	Week 2–3
Owner	26MML0006
Goal	Stages are idempotent, commit atomically to MinIO, checkpoint according to the policy, and a re-run with the same job id resumes.
Files	rds/policy.py, rds/stage.py, rds/events.py, rds/cli.py, tests/test_policy.py

Tasks

1. Read and understand `rds/policy.py` (never / always / adaptive Young-Daly) and run `tests/test_policy.py`.
2. Understand `rds/stage.py`: restore-if-committed → compute → persist+count → policy → write data → write `_RDS_COMMIT` marker.
3. Manual resume test: start `etl_taxi` with `--policy always`, kill the driver after the second stage (`python -m chaos.inject driver_crash` via API or `docker compose restart controller`), resubmit with the same `--job-id` → events must show `stage_restored`.
4. Check the event log `results/events/<job_id>.jsonl` contains `stage_completed` with `compute_s`, `write_s`, `reason`, `tau_s`.
5. Verify linear-chain skipping on `iterative_rank` (later iteration committed → earlier ones `stage_skipped`).

Commands

```
python -m pytest tests/test_policy.py -q
docker compose exec controller python -m controller.cli submit iterative_rank --mode smart --policy
adaptive
```

Done when

- ☐ Resumed run shows restored stages and produces the same output as an uninterrupted run.
- ☐ Checkpoint decisions in the event log match τ* from the formula.

P4 — Resilience controller (C1–C5)

When	Week 3–4
Owner	26MML0054 (C1, C5), 26MML0015 (C3, C4, core)
Goal	The controller detects a failed cluster, explains why, and moves stalled jobs to a healthy cluster where they resume.
Files	controller/*.py, configs/clusters.yaml, tests/test_detection.py, tests/test_reliability_and_jobs.py, tests/test_controller_sim.py

Tasks

1. C1 – `monitor.py`, `anomaly.py`, `health.py`, `statemachine.py`: confirm health 1.0 when idle; stop master A and watch H drop to 0.30 and state go SUSPECT → FAILED (3 polls).
2. C3 – `jobs.py`: spark-submit in client mode, process-group kill, retry vs failover rules (`on_attempt_exit`, `while_running`).
3. C4 – `orchestrator.py`: placement cost J; test with one cluster busy and one idle.
4. C5 – `decisions.py`, `reliability.py`: decision log entries and reliability score in `GET /clusters`.
5. `core.py` + `api.py` + `cli.py`: control loop every 2 s, REST API, Prometheus metrics; Grafana panels move during a fault.
6. Live failover demo: submit `etl_taxi --mode smart`, inject `cluster_outage` on A after ~40 s, verify job ends SUCCEEDED on `A>B` with restored stages.

Commands

```
docker compose exec controller python -m controller.cli submit etl_taxi --mode smart --policy
adaptive
docker compose exec controller python -m controller.cli chaos cluster_outage --cluster A
docker compose exec controller python -m controller.cli decisions --limit 20
docker compose exec controller python -m controller.cli restore
```

Done when

- ☐ Decision log shows transition A→FAILED with causes, a failover A→B with all J_k, and job_end SUCCEEDED.
- ☐ Master-only kill produces a `keep` decision (job not migrated).
- ☐ Recorded as execution video #3.

P5 — Fault injection (chaos harness)

When	Week 4
Owner	26MML0006
Goal	All nine faults can be injected and fully restored, repeatably.
Files	chaos/inject.py, controller/api.py (/chaos endpoints)

Tasks

1. Test each fault manually on an idle cluster and observe the effect: executor_kill, worker_kill, master_kill, cluster_outage, network_partition, network_delay, straggler, disk_full, driver_crash.
2. Check transient faults auto-restore after `--duration` (60 s) and that `restore` brings both clusters back to 2 ALIVE workers.
3. Check `tc` works (needs `cap_add: NET_ADMIN`) and that network reconnect restores the hostname alias.
4. Note the observed controller reaction per fault (state reached, time) — this becomes text for Section XV.

Commands

```
docker compose exec controller python -m chaos.inject network_delay --cluster A --duration 60
docker compose exec controller python -m chaos.inject restore
```

Done when

- ☐ A fault/observation table for all 9 faults.
- ☐ After every restore, `controller.cli status` shows both clusters HEALTHY within 30 s.

P6 — Experiments

When	Week 5–6
Owner	26MML0006 (run), 26MML0054 (monitor data quality)
Goal	Raw, reproducible measurements for all configurations, workloads and faults.
Files	experiments/run_matrix.py, experiments/configs/smoke.yaml, experiments/configs/full.yaml, results/raw/

Tasks

1. Smoke test: `experiments/configs/smoke.yaml` (B1 vs P, etl_taxi, none + cluster_outage). Expected: B1 FAILED at deadline, P SUCCEEDED on A>B.
2. Calibrate `full.yaml`: `fault_at_s` ≈ 40 % of each clean runtime, `expected_runtime_s` = measured clean runtime, deadlines ≈ 4× clean runtime.
3. Dry run to see the size and time estimate; reduce `repetitions` or faults if time is short (minimum: 3 reps of none, cluster_outage, master_kill, executor_kill, network_delay).
4. Run the full matrix in the background; check `runs.csv` every few hours for RUNNER_ERROR rows and fix causes.
5. Run the two case studies with a cluster outage (manual or a small yaml).
6. Freeze the results: copy `results/raw/full/` (without logs) to the repository.

Commands

```
docker compose exec controller python -m experiments.run_matrix experiments/configs/smoke.yaml
docker compose exec controller python -m experiments.run_matrix experiments/configs/full.yaml --dry-run
docker compose exec -d controller python -m experiments.run_matrix experiments/configs/full.yaml
```

Done when

- ☐ Smoke result as expected.
- ☐ `runs.csv` contains all planned runs with no RUNNER_ERROR (or documented re-runs).
- ☐ experiment.json saved with exact configuration.

P7 — Analysis, figures, paper integration and deliverables

When	Week 7–8
Owner	26MML0054 (analysis), 26MML0047 (paper), 26MML0015 (integration, videos)
Goal	Results turned into Tables 11–15 and Figures 21–24/28; manuscript completed; deliverables submitted.
Files	analysis/*.py, paper/content/*.txt, paper/build_paper.py, paper/make_figures.py

Tasks

1. Run `analysis.report` → Table 11/12 CSV+MD, Fig 21–24 (600 DPI), runs_enriched.csv.
2. Decide each hypothesis H1–H5 (p-value, Cliff's δ); write Section XV–XVII, XIX text; fill all yellow `[[...]]` in `paper/content/*.txt`.
3. Copy Fig 21–24 into `paper/Diagrams/` with the same names; draw Fig 28 (benefit sensitivity); rebuild the manuscript.
4. Record the five execution videos (README §11); push code, configs, dataset scripts and raw results to GitHub; add the link to Data Availability.
5. Similarity check (≤ 10 %), final proof-read, Word `Ctrl+A, F9` to refresh page numbers, submit.

Commands

```
docker compose exec controller python -m analysis.report results/raw/full --out results/report
python paper/make_figures.py
python paper/build_paper.py
```

Done when

- ☐ No `[[` left in the manuscript.
- ☐ All deliverables of the guidelines uploaded (manuscript, diagrams, code link, datasets, videos).

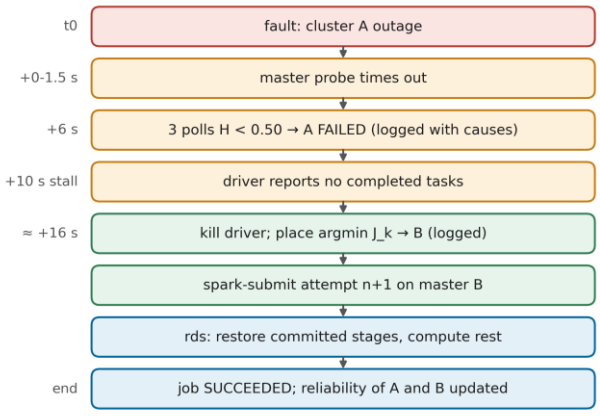


Figure 2. Behaviour to demonstrate at the end of P4 (failover workflow).

5. Module Specifications

Use this section if you rebuild a module yourself. Keep the public interface identical so the tests and the other modules keep working.

Module	Responsibility	Key interface	Tests
rds/policy.py	Checkpoint decision	`young_interval(C,M)`, `daly_interval(C,M)`, `AdaptivePolicy(mtbf).decide(work, cost) -> Decision(checkpoint, reason, interval_s)`, `make_policy(name, mtbf)`	test_policy.py
rds/stage.py	Resumable stages	`ResilientPipeline(spark, ctx, linear=False)`; decorator `@pipe.stage(name, kind='dataframe'|'model', loader=None)`; `pipe.finish()`; marker `<root>/<job>/<stage>/_RDS_COMMIT`	manual resume test (P3)
rds/cli.py	Job arguments	`base_parser()` adds --job-id --attempt --cluster --policy --mtbf --checkpoint-root --data-root --results-root --events-dir	—
controller/anomaly.py	EWMA z-score	`EwmaZScore(alpha, warmup).update(x, z_gate) -> z`; `ramp(x, low, high)`	test_detection.py
controller/health.py	Health score	`HealthModel(weights, expected_workers, ...).assess(Observation) -> HealthReport(health, anomalies, contributions, critical)`	test_detection.py
controller/statemachine.py	Failure detectors	`HealthStateMachine(cluster, suspect, failed, n_fail, n_rec).update(ts, H, causes)`; `TimeoutDetector(cluster, timeout).update(ts, reachable)` → `Transition`	test_detection.py
controller/orchestrator.py	Placement	`place(views, alpha, beta, gamma, exclude) -> Placement(cluster, scores, reason)`	test_detection.py
controller/reliability.py	Reliability	`ClusterReliability.observe(ts, failed, health)`, `.record_job(ok)`, `.score(weights, mttr_ref)`; `system_mtbf(trackers)`	test_reliability_and_jobs.py
controller/jobs.py	Driver processes	`JobSpec`, `JobManager.start_attempt / kill_attempt / crash_driver / refresh_progress`; pure rules `on_attempt_exit(...)`, `while_running(...)`	test_reliability_and_jobs.py, test_controller_sim.py
controller/core.py	Control loop	`Controller(cfg).submit(JobSpec)`, `.tick(now)`, `.run_forever(stop)`, `.snapshot()`	test_controller_sim.py
controller/api.py	REST + metrics	GET /clusters /jobs /decisions /metrics; POST /jobs /chaos /chaos/restore	manual (P4)
chaos/inject.py	Faults	`FaultInjector(cfg).inject(fault, cluster, duration_s, controller, job_id)`, `.restore_all()`	manual (P5)
experiments/run_matrix.py	Matrix runner	reads YAML → writes runs.csv, decisions.jsonl, cluster_timeseries.csv, events/, logs/	smoke run (P6)
analysis/report.py	Results	`python -m analysis.report <raw_dir> --out <dir>` → Tables 11/12, Figs 21–24	test_analysis.py + real data

Key configuration (configs/clusters.yaml)

Key	Value	Meaning
poll_interval_s / probe_timeout_s	2.0 / 1.5	Control-loop period; master probe timeout
health_weights	unreachable 0.45, workers_lost 0.25, latency 0.10, stall 0.15, executor_loss 0.05	Weights of H (sum normalised to 1)
latency_ewma_alpha, z_low, z_high	0.2, 3, 8	Latency anomaly ramp
stall_limit_s	120	No completed task for 120 s → stall anomaly 1
suspect / failed threshold	0.95 / 0.50	SUSPECT below 0.95; FAILED after 3 polls below 0.50
recovered_consecutive	5	Polls ≥ 0.95 before HEALTHY again

Key	Value	Meaning
migrate_stall_s	10	Stall needed before a job leaves a FAILED cluster
naive_timeout_s	30	Baseline B3 timeout
placement α, β, γ	0.4, 0.5, 0.1	Load, health, transfer weights
mtbf_prior_s / weight	900 / 1	Prior of the online MTBF estimate
spark_submit	driver 512m; executor 576m, 1 core; 4 cores/job	Per-job resources

6. Testing Strategy

Level	What	How	When
Unit	Formulas, health model, state machine, placement, reliability, recovery rules, metrics, statistics	`python -m pytest` (56 tests, < 5 s, no Docker)	Every commit / PR
Simulation	Whole controller with fake clock, probes and drivers: outage, timeout baseline, keep decision, explanations	tests/test_controller_sim.py	Every PR touching controller/
Integration	Real Spark + MinIO: job runs, resumes, fails over	P3 resume test, P4 live demo, smoke.yaml	End of P3, P4, start of P6
Fault validation	Each of 9 faults has the expected effect and is restored	P5 checklist	End of P5
Experiment sanity	No RUNNER_ERROR, clean runs stable, deadlines not hit by healthy runs	Inspect runs.csv, decisions.jsonl	Daily during P6

Rule: a pull request is merged only if `python -m pytest` passes and, for changes in controller/ or rds/, the smoke experiment still gives B1 FAILED / P SUCCEEDED.

7. Experiment Protocol

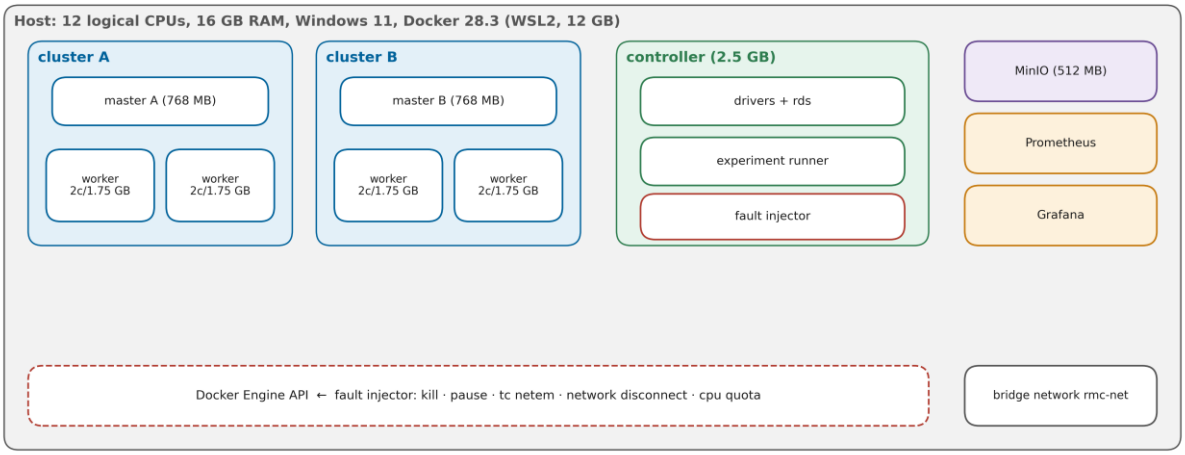


Figure 3. Experimental testbed (Fig. 19 of the paper).

Item	Setting
Systems	B1 vanilla (none/never) · B2 fixed checkpoint (none/always) · B3 naive HA (naive/never) · A1 ablation (smart/always + speculation) · P RMC-Spark (smart/adaptive + speculation)
Workloads	etl_taxi · ml_taxi · iterative_rank · streaming (+ case_fraud, case_maintenance)
Faults	none · executor_kill · worker_kill · master_kill · cluster_outage · network_partition · network_delay · straggler (transient faults reverted after 60 s)
Repetitions	3 per cell, shuffled within each repetition (seed 2026) → 480 runs
Fault time	≈ 40 % of the clean runtime of the workload, on the job's current cluster; every run starts on cluster A
Between runs	restore_all → wait until both clusters HEALTHY → 10 s cool-down
Outputs	results/raw/full/: runs.csv, decisions.jsonl, cluster_timeseries.csv, events/*.jsonl, logs/, experiment.json
Metrics	success rate, duration, delay, downtime, availability, detection & failover latency, recomputed work, checkpoint cost, overhead, utilisation, streaming recovery
Statistics	Mann-Whitney U (two-sided), Cliff's δ, 95 % bootstrap CI (5000 resamples)

Expected results to check (hypotheses)

#	Expectation	If not observed
H1	P completes more jobs than B1/B2 under cluster outage and partition	Check failover in decisions.jsonl; check deadline long enough
H2	P recovers faster than B3	Check stall rule (10 s) and detection (3 polls) timings
H3	Master-only kill: P keeps the job, B3 restarts it	Check driver progress probing (ui port, REST)
H4	Less recomputed work for P/A1 on iterative_rank	Check committed stages and linear skipping
H5	P overhead < A1 and B2 in clean runs	Report honestly; discuss checkpoint cost vs τ*

8. Timeline and Responsibilities (8 weeks)

Week	26MML0015 (PoC)	26MML0054	26MML0006	26MML0047
1	P0 repo, conventions, review	P0 setup; study C1 code	P0 + P1 testbed	P0; literature check, datasets README
2	Study core/jobs; API demo	C1 tests + Grafana panels	P2 workloads, clean runtimes	P2 case-study downloads + runs
3	P4 C3/C4 live failover	P4 C1/C5 validation	P3 rds resume tests	Security controls (optional spark.authenticate)
4	P4 integration, video #3	Reliability score check	P5 fault validation	Section XII/XVIII drafting
5	Monitor matrix	Data-quality checks on runs.csv	P6 smoke + calibrate + start full	Case studies with outage
6	Fix runner issues, re-runs	Start analysis on partial data	P6 full matrix completion	Section XVII drafting
7	Videos, GitHub release	P7 analysis, Tables 11–12, Figs 21–24	Appendix configs	Fill results text, Fig 28
8	Final integration, submission	Results text Sections XV, XIX	Section XIII/XVI final	Similarity check, references, proof-read

Compressed 3-week schedule (if the deadline is near)

Week	Must finish
1	P0 + P1 + P2 (all workloads clean) + P3 resume test
2	P4 live failover + P5 quick fault check + P6 reduced matrix: 5 systems × 2 workloads (etl_taxi, iterative_rank) × 4 faults (none, cluster_outage, master_kill, network_delay) × 3 reps = 120 runs (≈ 10–12 h)
3	P7: analysis, fill manuscript, videos, GitHub, submission

9. Risks and Troubleshooting

Risk / symptom	Impact	Mitigation
Containers exit 137 (out of memory)	Runs fail	12 GB WSL; lower worker `--memory/mem_limit`; close other apps
`dockerDesktopLinuxEngine` connect error	Nothing runs	Start Docker Desktop; wait for 'Engine running'
ClassNotFoundException S3AFileSystem	No storage	`docker compose build --no-cache` (jars missing)
Driver UI port in use (4040–4049)	Submit fails	Do not use the API while the matrix runs; `docker compose restart controller`
Workers not re-registering after restore	Next run unhealthy	Runner waits for HEALTHY; else restart worker containers
Streaming job ends immediately	Wrong result	Use a new job id or delete its checkpoint in MinIO
Matrix takes too long	Late results	Use the compressed matrix (Section 8); run overnight with `-d`
Laptop sleeps during runs	Broken runs	Disable sleep; keep charger connected
Noisy timings on shared host	Weak statistics	Close other apps; shuffle order (already); report medians
Results do not support a hypothesis	Weaker paper	Report honestly and explain — negative results are acceptable

10. Definition of Done and Deliverables

- ☐ All 56 unit/simulation tests pass on `main`.
- ☐ Live failover demo works and is recorded.
- ☐ All nine faults validated and restorable.
- ☐ Full (or compressed) matrix completed; raw data committed to GitHub.
- ☐ Tables 11–15 and Figures 21–24, 28 generated from data; manuscript has no `[[` placeholders.
- ☐ Deliverables (guidelines): manuscript .docx, Diagrams folder, GitHub link, datasets folder/scripts, execution videos, data availability statement.
- ☐ Similarity ≤ 10 %; references unchanged or re-verified.

Command cheat sheet

```
# unit tests (host)
python -m pytest
# testbed
docker compose build
docker compose up -d
docker compose ps
docker compose down        # add -v to wipe MinIO data
# data
docker compose exec controller spark-submit --master spark://spark-master-a:7077 --conf
spark.driver.host=controller --total-executor-cores 4 --executor-memory 576m jobs/datagen.py
# controller
docker compose exec controller python -m controller.cli status
docker compose exec controller python -m controller.cli submit etl_taxi --mode smart --policy
adaptive
docker compose exec controller python -m controller.cli chaos cluster_outage --cluster A
docker compose exec controller python -m controller.cli decisions --limit 20
docker compose exec controller python -m controller.cli restore
# experiments
docker compose exec controller python -m experiments.run_matrix experiments/configs/smoke.yaml
docker compose exec -d controller python -m experiments.run_matrix experiments/configs/full.yaml
# analysis + paper
docker compose exec controller python -m analysis.report results/raw/full --out results/report
python paper/make_figures.py
python paper/build_paper.py
```